

Solver Specification:

highOrderElectroActivationFoamImplicitJfNK_PETSc

A high-order, implicit, Jacobian-free Newton–Krylov monodomain solver in OpenFOAM with PETSc back-end

Pablo

May 12, 2026

Abstract

This report specifies the algorithms implemented in the OpenFOAM solver **highOrderElectroActivationFoamImplicitJfNK_PETSc**. The solver advances the monodomain equations of cardiac electrophysiology coupled to the ten Tusscher–Noble–Noble–Panfilov (TNNP, 2004) ionic cell model. The diffusion operator is discretised with the cell-centred finite-volume framework of OpenFOAM, optionally upgraded to a Local Radial-Extrapolation (LRE) high-order reconstruction; the ionic source is evaluated either at cell centres or at LRE cell quadrature points; time integration follows a θ -scheme (backward Euler or Crank–Nicolson); the resulting nonlinear coupled system is solved by a Jacobian-Free Newton–Krylov (JFNK) method with Armijo backtracking line search, linear extrapolation initial guess and a physical-bracket clamp on the input to the ionic ODE; the inner linear system is solved by PETSc’s KSP framework (GMRES with modified Gram–Schmidt by default); the per-cell ionic ODE uses OpenFOAM’s adaptive RKF45 integrator with configurable tolerances; activation times are tracked by linear interpolation across the first threshold crossing and benchmark points are sampled by inverse-distance weighting. All algorithmic parameters are exposed through the **constant/spatialIntegrationProperties** and **constant/timeIntegrationProperties** dictionaries, and the companion Python driver **run_convergence.py** generates these dictionaries from a single configuration file.

Contents

1	Notation	2
2	Governing equations	3
2.1	Monodomain PDE	3
2.2	Ionic ODE	3
2.3	External stimulus	3
3	Time discretisation: the θ-scheme	3
4	Spatial discretisation	4
4.1	Standard orthogonal FV Laplacian	4
4.2	High-order LRE diffusion	4
4.3	Mass matrices	5
4.4	High-order reconstruction at ionic quadrature points	5
4.5	High-order cell averaging of I_{ion}	5
5	Nonlinear solver	5
5.1	Jacobian-Free Newton–Krylov (JFNK)	6
5.2	Armijo backtracking line search	6
5.3	Linear extrapolation initial guess	6
5.4	Protective Vm clamp on ODE input	6

5.5	Convergence criteria	7
5.6	Rollback on non-convergence	7
5.7	Alternative nonlinear methods	7
6	Linear solver: PETSc KSP	7
7	Ionic ODE integration	8
8	TNNP numerical protection	8
9	Activation tracking and benchmarks	9
9.1	Activation time per cell	9
9.2	Early termination on benchmark point	9
9.3	Diagonal sampling	9
10	Solver flow chart	10
11	Driver script: run_convergence.py	12
11.1	Geometry and parameter sweep	12
11.2	Nonlinear solver (JFNK / Picard)	12
11.3	JFNK matrix-free options	13
11.4	Linear solver	13
11.5	Inner ODE integrator (RKF45)	13
11.6	LRE high-order parameters per p-level	14
11.7	Post-processing and run-control	14
A	Dimension table	14
B	File map	15

1 Notation

Symbol	Meaning	Units
$\Omega \subset \mathbb{R}^d$	Spatial domain ($d \in \{2, 3\}$)	m
$\partial\Omega$	Insulated boundary (zero-flux Neumann)	—
$t \in (0, T]$	Time	s
$V_m(\mathbf{x}, t)$	Transmembrane voltage	V
$\mathbf{s}(\mathbf{x}, t) \in \mathbb{R}^{17}$	TNNP ionic states	mixed
$I_{\text{ion}}(V_m, \mathbf{s})$	Total ionic current density	V s^{-1}
$I_{\text{ext}}(\mathbf{x}, t)$	External stimulus current density	A m^{-3}
χ	Membrane surface-to-volume ratio	m^{-1}
C_m	Membrane capacitance per unit area	F m^{-2}
$\mathbf{D}(\mathbf{x})$	Bulk conductivity tensor	S m^{-1}
$\theta \in \{0.5, 1\}$	Time-stepping parameter (CN or BE)	—
Δt	Outer time-step length	s
N_c	Number of mesh cells	—
$\mathbf{V}^n \in \mathbb{R}^{N_c}$	Vector of cell-averaged V_m at time t^n	V
$\mathbf{M}, \mathbf{K}, \mathbf{L}$	Mass, stiffness, scaled diffusion matrix	—
$\mathbf{A}, \mathbf{B}$	Implicit LHS / RHS operators of the θ -scheme	—
$\mathbf{F}(\mathbf{x})$	Discrete nonlinear residual to be zeroed	—

2 Governing equations

2.1 Monodomain PDE

The bulk voltage is propagated by the standard monodomain reaction–diffusion equation [1, 2]:

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - I_{\text{ion}}(V_m, \mathbf{s}) + I_{\text{ext}} \quad \text{in } \Omega \times (0, T], \quad (1)$$

subject to insulated boundaries

$$\mathbf{n} \cdot (\mathbf{D} \nabla V_m) = 0 \quad \text{on } \partial\Omega \times (0, T], \quad (2)$$

and initial condition $V_m(\mathbf{x}, 0) = -84 \text{ mV}$. The conductivity tensor $\mathbf{D}$ is assumed symmetric positive-definite and either uniform or read as a `volTensorField` from disk.

Dividing (1) by χC_m gives the form actually used inside the solver,

$$\frac{\partial V_m}{\partial t} = \mathcal{L} V_m + s(V_m, \mathbf{s}), \quad \mathcal{L} = \frac{1}{\chi C_m} \nabla \cdot (\mathbf{D} \nabla \cdot), \quad s = -\frac{I_{\text{ion}}}{\chi C_m} + \frac{I_{\text{ext}}}{\chi C_m}. \quad (3)$$

2.2 Ionic ODE

At every spatial sampling point (cell centre or LRE cell quadrature point) the ionic state vector $\mathbf{s} \in \mathbb{R}^{17}$ evolves according to the TNNP 2004 system

$$\frac{d\mathbf{s}}{dt} = \mathbf{f}_{\text{TNNP}}(V_m, \mathbf{s}; \text{tissue}), \quad \mathbf{s}(0) = \mathbf{s}_0, \quad (4)$$

where V_m is supplied externally by the PDE solver, and `tissue` selects one of `epicardialCells`, `mCells` or `endocardialCells`. The 17 states tracked in `TNNP_2004Names.H` are

$$\mathbf{s} = (V, K_i, Na_i, Ca_i, Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g, Ca_{SR}).$$

The total ionic current I_{ion} returned to the PDE is the algebraic quantity `Iion.cm` (sum of $i_{Na}, i_{K1}, i_{to}, i_{Kr}, i_{Ks}, i_{CaL}, i_{NaK}, i_{NaCa}, i_{pK}, i_{pCa}, i_{bNa}, i_{bCa}$). See the original publication [1] for the algebraic definitions of each current.

2.3 External stimulus

The external current I_{ext} is supplied as a time-windowed, region-windowed step,

$$I_{\text{ext}}(\mathbf{x}, t) = \begin{cases} I_0 & \mathbf{x} \in \Omega_{\text{stim}}^{(b)}, \quad t \in [t_0^{(b)}, t_0^{(b)} + \tau], \quad b = 1, \dots, B, \\ 0 & \text{otherwise,} \end{cases} \quad (5)$$

with stimulus boxes $\Omega_{\text{stim}}^{(b)}$, start times $t_0^{(b)}$, common amplitude I_0 and duration τ read from `constant/stimulusProtocol`. Default Niederer-benchmark values are $I_0 = 50\,000 \text{ A m}^{-3}$ and $\tau = 2 \text{ ms}$.

3 Time discretisation: the θ -scheme

The semi-discrete equation (3) is advanced from t^n to $t^{n+1} = t^n + \Delta t$ by a one-parameter family

$$\frac{\mathbf{V}^{n+1} - \mathbf{V}^n}{\Delta t} = \theta \mathbf{L} \mathbf{V}^{n+1} + (1 - \theta) \mathbf{L} \mathbf{V}^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n. \quad (6)$$

Two values of θ are selectable by the dictionary key `implicitScheme`:

<code>implicitScheme</code>	θ	Scheme
<code>backwardEuler</code>	1	First-order, L-stable, fully implicit
<code>crankNicolson</code>	1/2	Second-order, A-stable (not L-stable)

Multiplying (6) by Δt^{-1} on both sides yields the linear system actually assembled,

$$\begin{aligned} [\mathbf{M}/\Delta t - \theta \mathbf{L}] \mathbf{V}^{n+1} &= [\mathbf{M}/\Delta t + (1 - \theta) \mathbf{L}] \mathbf{V}^n \\ &+ \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n. \end{aligned} \quad (7)$$

We name the two PETSc Mat operators in (7)

$$\mathbf{A} = \mathbf{M}/\Delta t - \theta \mathbf{L}, \quad \mathbf{B} = \mathbf{M}/\Delta t + (1 - \theta) \mathbf{L}. \quad (8)$$

They are rebuilt at every time step by duplicating $\mathbf{M}$, scaling by Δt^{-1} and adding $\pm \theta \mathbf{L}$ via `MatAXPY` with the `DIFFERENT_NONZERO_PATTERN` flag (since $\mathbf{M}$ and $\mathbf{L}$ have disjoint sparsity patterns in general). When $\theta = 1$ (BE) the explicit half of the diffusion vanishes and the second `MatAXPY` call is skipped, leaving $\mathbf{B} = \mathbf{M}/\Delta t$.

The scaled diffusion operator $\mathbf{L}$ is itself derived from the stiffness matrix $\mathbf{K}$ by

$$\mathbf{L} = \frac{1}{\chi C_m} \mathbf{K}, \quad (9)$$

assembled once at solver start-up by `MatDuplicate + MatScale` on a deep copy of $\mathbf{K}$.

4 Spatial discretisation

The mesh is the standard OpenFOAM `fvMesh`. All operators below return PETSc Mat objects of type `MATSEQAIJ` with preallocated non-zero counts per row.

4.1 Standard orthogonal FV Laplacian

`assembleStandardOrthogonalStiffnessMatrix` computes the classical 5/7-point Laplacian. For each internal face f between cells `own` and `nei` with outward (from `own`) face-area vector $\mathbf{S}_f$ and cell-centre separation $\mathbf{d}_f = \mathbf{C}_{\text{nei}} - \mathbf{C}_{\text{own}}$:

$$a_f = \frac{|\mathbf{S}_f| (\mathbf{n}_f \cdot \mathbf{D}_f \cdot \hat{\mathbf{d}}_f)}{|\mathbf{d}_f|}, \quad \mathbf{D}_f = \frac{1}{2} (\mathbf{D}_{\text{own}} + \mathbf{D}_{\text{nei}}), \quad \hat{\mathbf{d}}_f = \mathbf{d}_f / |\mathbf{d}_f|. \quad (10)$$

Per face the four matrix contributions assembled with `ADD_VALUES` into $\mathbf{K}$ are

$$\begin{aligned} \mathbf{K}[\text{own}, \text{own}] &+= -a_f / V_{\text{own}}, & \mathbf{K}[\text{own}, \text{nei}] &+= +a_f / V_{\text{own}}, \\ \mathbf{K}[\text{nei}, \text{own}] &+= +a_f / V_{\text{nei}}, & \mathbf{K}[\text{nei}, \text{nei}] &+= -a_f / V_{\text{nei}}. \end{aligned} \quad (11)$$

Faces on a `zeroGradient` or `empty` patch contribute zero flux. Faces on a `fixedValue` or `fixedVoltage` patch contribute only to the owner diagonal; the prescribed value enters the right-hand side via OpenFOAM's boundary-correction mechanism. The sparsity is preallocated at 7 non-zeros per row, sufficient for any hexahedral / unstructured mesh up to six face neighbours.

4.2 High-order LRE diffusion

When `useHighOrder_Vm` is `true` and the geometric dimension exceeds one, `assembleHighOrderStiffnessMatrix` is used instead. It performs Gauss quadrature on each face with reconstructed gradients provided by the LRE library:

$$\Phi_f = \sum_{q \in \mathcal{Q}_f} w_q |\mathbf{S}_f| \mathbf{n}_f \cdot \mathbf{D}_f \cdot \underbrace{\left(\sum_{j \in \mathcal{S}_f} \mathbf{g}_{f,q,j} \mathbf{V}_j \right)}_{\nabla V_m|_q}, \quad (12)$$

where $\mathcal{Q}_f$ are the face quadrature points, $\mathcal{S}_f$ is the LRE face stencil (typically 15–50 cells) and $\mathbf{g}_{f,q,j}$ are the `QRGradFaceGPCoeffs` returned by the LRE object. The contribution of each $\mathbf{V}_j$ to the flux is added to row `own` with weight $+\Phi_{f,j}/V_{\text{own}}$ and to row `nei` with weight $-\Phi_{f,j}/V_{\text{nei}}$, giving the expected negative semi-definite Laplacian. Preallocation is 60 non-zeros per row.

4.3 Mass matrices

Two mass matrices are available:

Lumped (default). `assembleDiagonalMassMatrix` writes $\mathbf{M} = I$ on the cell-centre basis,

$$\mathbf{M}[i, i] = 1, \quad i = 1, \dots, N_c, \quad (13)$$

preallocated at one non-zero per row.

Consistent high-order. `assembleConsistentMassMatrixHO` fills a multi-stencil mass matrix by Gauss quadrature on each cell. For every cell i and every stencil neighbour $j \in \mathcal{S}_i \cup \{i\}$,

$$\mathbf{M}[i, j] = \sum_{q \in \mathcal{Q}_i} \frac{w_q}{\sum_{q'} w_{q'}} \phi_j(\mathbf{x}_q), \quad (14)$$

where ϕ_j is the LRE Taylor reconstruction basis function of neighbour j at quadrature point $\mathbf{x}_q$:

$$\phi_j(\mathbf{x}_q) = \delta_{ij} + \mathbf{g}_{i,j} \cdot \mathbf{d}_q + \frac{1}{2} \mathbf{H}_{i,j} : (\mathbf{d}_q \otimes \mathbf{d}_q) + \frac{1}{6} \mathbf{T}_{i,j}^{(3)} (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \quad (15)$$

with $\mathbf{d}_q = \mathbf{x}_q - \mathbf{C}_i$. The gradient, Hessian and 3rd-order tensor coefficients are `QRGradCoeffs`, `cellHessianCoeffs` and `cellThirdDerivCoeffs` respectively, and the cubic term is contracted by `LRE::cubicForm`. The weight normalisation in (14) guarantees that $(\mathbf{M} \mathbf{1})_i = 1$ for constant fields (consistency on constants is exact at any order).

A safety guard at line 1500 of the source forces $\mathbf{M}$ to the lumped form whenever `useHighOrder_Vm` = `false` or $d \leq 1$, regardless of `massMatrix`, with an informational message.

4.4 High-order reconstruction at ionic quadrature points

When `useHighOrder_Iion` is `true` the ionic ODE is integrated at cell quadrature points rather than at cell centres. The voltage is reconstructed there by the same Taylor expansion as in (15),

$$V_m(\mathbf{x}_q) \approx V_{m,c} + (\nabla V_m)_c \cdot \mathbf{d}_q + \frac{1}{2} \mathbf{H}_c : (\mathbf{d}_q \otimes \mathbf{d}_q) + \frac{1}{6} \mathbf{T}_c^{(3)} (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \quad (16)$$

implemented in `reconstructVmAtIionIntegrationPoints`. The order of (16) is the one of the V_m LRE object, independently of the LRE object used for I_{ion} integration. If `useHighOrder_Vm` is `false` the linear fallback $V_{m,c} + (\nabla V_m)_c \cdot \mathbf{d}_q$ is used with $(\nabla V_m)_c$ from the standard OpenFOAM `fvc::grad`.

4.5 High-order cell averaging of I_{ion}

After the ODE solve, `averageIionIntegrationPointsToCells` reduces the per-quadrature-point ionic currents to cell averages by

$$I_{\text{ion},i} = \frac{\sum_{q \in \mathcal{Q}_i} w_q I_{\text{ion},q}}{\sum_{q \in \mathcal{Q}_i} w_q}, \quad (17)$$

which is the natural counterpart of the Gauss reconstruction in (16).

5 Nonlinear solver

The fully discrete equation (7) is nonlinear in $\mathbf{V}^{n+1}$ through $I_{\text{ion}}(\mathbf{V}^{n+1}, \mathbf{s}^{n+1})$, where $\mathbf{s}^{n+1}$ is itself obtained by integrating (4) from t^n to t^{n+1} with input $\mathbf{V}^{n+1}$. We define the discrete residual

$$\mathbf{F}(\mathbf{x}) = \mathbf{A} \mathbf{x} - \mathbf{r}(I_{\text{ion}}(\mathbf{x})), \quad \mathbf{x} = \mathbf{V}^{n+1}, \quad (18)$$

with $\mathbf{r}(I_{\text{ion}}) = \mathbf{B} \mathbf{V}^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n$, and seek the unique root $\mathbf{F}(\mathbf{x}) = 0$.

5.1 Jacobian-Free Newton–Krylov (JFNK)

Newton’s iteration is

$$\mathbf{J}(\mathbf{x}_k) \boldsymbol{\delta}_k = -\mathbf{F}(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \boldsymbol{\delta}_k, \quad (19)$$

with $\mathbf{J} = \partial \mathbf{F} / \partial \mathbf{x}$ and step length $\alpha_k \in (0, 1]$. The Jacobian $\mathbf{J}$ is never assembled. Instead, every matrix–vector product $\mathbf{J}(\mathbf{x}_k) \mathbf{v}$ required by GMRES is approximated by a forward finite difference [3],

$$\mathbf{J}(\mathbf{x}_k) \mathbf{v} \approx \frac{\mathbf{F}(\mathbf{x}_k + \varepsilon \mathbf{v}) - \mathbf{F}(\mathbf{x}_k)}{\varepsilon}, \quad \varepsilon = \varepsilon_0 \frac{1 + \|\mathbf{x}_k\|_2}{\|\mathbf{v}\|_2}, \quad (20)$$

where $\varepsilon_0 = \text{jfnkEpsilon}$. The textbook choice $\varepsilon_0 \sim \sqrt{\text{eps}_{\text{machine}}} \approx 10^{-8}$ is optimal only when $\mathbf{F}$ is computed in exact arithmetic; when $\mathbf{F}$ contains numerical noise of relative magnitude η (here $\eta \sim \text{relTol}$ of the inner RKF45 integrator), the optimum balancing truncation and noise is $\varepsilon_0 \approx \sqrt{\eta}$ [3]. With the default $\eta = 10^{-6}$ we use $\varepsilon_0 = 10^{-5}$.

The wrapper `matVec` of equation (20) is exposed to PETSc as a `MatShell` so that any KSP type can act on it. The inner GMRES solve uses modified Gram–Schmidt orthogonalisation, a restart cycle of `jfnkMaxKrylovIterations` and up to `jfnkMaxRestarts` restarts.

5.2 Armijo backtracking line search

The unconditional update $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_0 \boldsymbol{\delta}_k$ (with $\alpha_0 = \text{nonlinearRelaxation}$) is fragile near the steep i_{Na} sigmoid of the cardiac AP upstroke, where the linearisation (20) at $\mathbf{x}_k$ underestimates the true nonlinearity. We globalise Newton by Armijo backtracking [4]:

$$\alpha_k = \alpha_0 \cdot 2^{-\ell_k}, \quad \ell_k = \min \left\{ \ell \geq 0 \mid \|\mathbf{F}(\mathbf{x}_k + 2^{-\ell} \alpha_0 \boldsymbol{\delta}_k)\|_2^2 \leq (1 - 2c 2^{-\ell} \alpha_0) \|\mathbf{F}(\mathbf{x}_k)\|_2^2 \right\}. \quad (21)$$

The sufficient-decrease constant is $c = \text{jfnkArmijoC}$ (default 10^{-4}). Backtracking stops when the test succeeds, after `jfnkLineSearchMaxIter` trials, or when $2^{-\ell} \alpha_0 < \text{jfnkLineSearchAlphaMin}$; in the latter two cases the current trial is accepted and outer convergence checking handles the next step. Disable with `jfnkLineSearch false` to recover the legacy fixed-damping update.

5.3 Linear extrapolation initial guess

Initial guess `jfnkInitGuessOrder` selects between

$$\mathbf{x}_0 = \mathbf{V}^n \quad (\text{jfnkInitGuessOrder} = 0), \quad (22)$$

$$\mathbf{x}_0 = \mathbf{V}^n + \frac{\Delta t}{\Delta t_{\text{prev}}} (\mathbf{V}^n - \mathbf{V}^{n-1}) \quad (\text{jfnkInitGuessOrder} = 1, \text{ default}). \quad (23)$$

The order-1 extrapolation is cell-wise clamped to `[jfnkInitGuessVmMin, jfnkInitGuessVmMax]` to protect Newton from starting in an unphysical bracket. On the first time step there is no $\mathbf{V}^{n-1}$ and the solver falls back to order 0.

5.4 Protective Vm clamp on ODE input

When `jfnkClampODEInput` is `true` (default), the `Vm` vector passed to `TNNP::solveODE` inside every call to `evaluateNonlinearFields` is clamped element-wise to the same physical bracket. The clamp is applied to the working buffer (`VmIntegrationPoints` for the HO-Ion path, a local `scalarField` copy otherwise) so that neither `VmCandidate` nor any persistent field is mutated. The kink that this introduces in $\mathbf{F}$ outside the bracket is well outside the region where the true solution lives, and is filtered out at the outer level by the Armijo test.

5.5 Convergence criteria

A Newton iteration is declared converged when *all* of the following hold:

$$\|F(\mathbf{x}_k)\|_2 / \|\mathbf{x}_k\|_2 \leq \text{nonlinearTolerance}, \quad (24)$$

$$\|\mathbf{V}_k - \mathbf{V}_{k-1}\|_2 / \|\mathbf{V}_k\|_2 \leq \text{nonlinearVmTolerance}, \quad (25)$$

$$\|I_{\text{ion},k} - I_{\text{ion},k-1}\|_2 / \|I_{\text{ion},k}\|_2 \leq \text{nonlinearIionTolerance}, \quad (26)$$

$$(\text{opt.}) \quad \max_i \|\mathbf{s}_k^{(i)} - \mathbf{s}_{k-1}^{(i)}\|_2 / \|\mathbf{s}_k^{(i)}\|_2 \leq \text{nonlinearStatesTolerance}, \quad (27)$$

and the iteration index satisfies $k + 1 \geq \text{minNonlinearIterations}$. The last criterion is gated by the boolean `nonlinearRequireStatesConvergence`. The coupled residual $\|F\|/\|\mathbf{x}\|$ has a noise floor of order $\eta = \text{relTol}$ (RK45) because F itself is computed in finite precision through the ODE integrator. Tolerances below this floor are unattainable in practice.

5.6 Rollback on non-convergence

If the Newton loop exits with the maximum iteration budget `nonlinearIterations` reached and the convergence flag still false, the boolean `nonlinearAcceptUnconverged` decides between:

- **false** (default): restore $V_m, I_{\text{ion}}, \mathbf{s}$ to their values at the start of the time step, emit a `WarningInFunction`, and proceed; the time step effectively becomes a no-op.
- **true**: commit the unconverged iterate and proceed anyway.

5.7 Alternative nonlinear methods

For comparison and stress-testing, the solver also implements two alternatives to JFNK selectable by `nonlinearMethod`:

picard

Fixed-point on the lagged Iion source: each outer iteration assembles $\mathbf{A}\mathbf{x}^{(k+1)} = \mathbf{r}(I_{\text{ion}}(\mathbf{x}^{(k)}))$ and solves a single KSP; the update is damped by `nonlinearRelaxation`. No matrix-free Jacobian and no line search are used. Robust at small Δt , linearly convergent.

diagonalIion (or diagonal, localDiagonal)

Picard with the diagonal of $\partial I_{\text{ion}} / \partial V_m$ folded into the LHS before each linear solve. The diagonal derivative is evaluated by a one-sided finite difference of step `diagonalIionEpsilon`. The RHS is shifted accordingly so that the fixed point is preserved. Faster than pure Picard for stiff sources, no matrix-free Jacobian needed.

6 Linear solver: PETSc KSP

The wrapper `solveLinearKSP` drives any PETSc Krylov solver on either the explicit operator $\mathbf{A}$ (Picard / diagonal paths) or the matrix-free Jacobian shell (JFNK path). The configuration is read from `implicitHOCoeffs`:

Key	Maps to	Default
<code>linearSolver</code>	<code>KSPTYPE</code>	GMRES
<code>preconditioner</code>	<code>PCTYPE</code>	ILU
<code>tolerance</code>	relative tolerance	10^{-10}
<code>maxIterations</code>	iteration cap	2000
<code>jfnkMaxKrylovIterations</code>	GMRES restart cycle m	30
<code>jfnkMaxRestarts</code>	maximum outer restarts	3

Supported KSP types. GMRES, FGMRES, BiCGSTAB (alias BCGS), PREONLY (alias SparseLU, used together with a direct PC type).

Supported PC types. ILU, JACOBI, BJACOBI, LU (alias SparseLU), NONE, SOR, ASM. Any preconditioner that PETSc supports is also reachable via command-line overrides `-pc_*` thanks to the call to `KSPSetFromOptions`.

Orthogonalisation. For GMRES and FGMRES the wrapper explicitly selects `KSPGMRESModifiedGramSchmidtOrthogonal` which keeps the Krylov basis orthogonal to within $O(\varepsilon_m \kappa(A))$ instead of the classical $O(\varepsilon_m \kappa(A)^2)$ [5], relevant for the stiff Jacobians of cardiac upstrokes.

7 Ionic ODE integration

For each cell (or each cell quadrature point if `useHighOrderIion` is `true`) the TNNP ODE (4) is advanced from t^n to t^{n+1} by the standard OpenFOAM adaptive integrator chosen via the dictionary key `solver`. The default is RKF45, a 4-5 embedded Runge–Kutta–Fehlberg pair with PI step controller [6]. The keys read from `constant/timeIntegrationProperties` are:

Key	Meaning	Default
<code>solver</code>	Concrete ODE integrator	RKF45
<code>absTol</code>	Absolute tolerance per state	10^{-9}
<code>relTol</code>	Relative tolerance per state	10^{-6}
<code>maxSteps</code>	Cap on sub-steps per outer call	10^4
<code>initialODEStep</code>	Initial sub-step size hint (ms)	10^{-5}

The relative tolerance `relTol` sets the noise floor in $F(x)$ and therefore the lowest attainable coupled residual in the JFNK loop. Tightening `relTol` buys lower outer tolerance at the cost of more RKF45 sub-steps per Newton iteration. The cap `maxSteps` guards against runaway integrations triggered by unphysical JFNK probes; combined with the Vm clamp of section 5.4 it produces a clear `FatalError` when something is truly wrong, rather than an indefinite hang.

8 TNNP numerical protection

The TNNP cell model contains several algebraic singularities (i_{CaL} at $V = 0$ mV, divisions by gate variables that should be in $[0, 1]$, and stiff exponentials of unbounded V). The solver wraps every external call into `TNNP::solveODE`, `TNNP::derivatives` and `TNNP::calculateCurrent` with a `protectState` call that enforces physically reasonable bounds *before* the algebraic kernel is evaluated.

The master switch and per-guard knobs are read by the TNNP constructor (TNNP.C lines 74–94) from the merged solver dictionary:

Key	Type	Default
<code>TNNPNumericalProtection</code>	Switch	<code>false</code>
<code>TNNPCLampGates</code>	Switch	<code>true</code>
<code>TNNPCLampVmForRates</code>	Switch	<code>true</code>
<code>TNNPLogNumericalProtection</code>	Switch	<code>true</code>
<code>TNNPConcentrationFloor</code>	scalar (mM)	10^{-12}
<code>TNNPVMinForRates</code>	scalar (mV)	-200
<code>TNNPVMaxForRates</code>	scalar (mV)	$+100$
<code>TNNPVoltageZeroEps</code>	scalar (mV)	10^{-8}
<code>TNNPMaxProtectionLogEntries</code>	label	10^6

When `TNNPNumericalProtection` is true, `protectState` applies the following checks in order, recording every correction in the in-memory `ProtectionSummaryEntry` table:

1. **Non-finite V .** If `!std::isfinite(state[V])`, V is reset to the resting potential -86.2 mV. Reason tag `nonfinite_V`.
2. **Vm clamping for rates** (only if `TNNPCLampVmForRates`). If $V < \text{TNNPVMinForRates}$ clamp to that floor; if $V > \text{TNNPVMaxForRates}$ clamp to that ceiling. Reason tags `Vm_below_minimum`, `Vm_above_maximum`.
3. **Avoid the i_{CaL} singularity at $V = 0$.** If $|V| < \text{TNNPVoltageZeroEps}$, push V to $\pm \text{TNNPVoltageZeroEps}$ preserving the original sign. Reason tag `avoid_i_CaL_V_zero_singularity`.
4. **Concentration floor.** For each ionic concentration in $\{K_i, Na_i, Ca_i, Ca_{SR}\}$: non-finite entries are reset to `TNNPConcentrationFloor`; values below the floor are pushed to it. Reason tags `nonfinite_concentration`, `concentration_below_floor`.
5. **Gate clamping** (only if `TNNPCLampGates`). For each gate in $\{Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g\}$: non-finite entries are reset to 0; values < 0 are clamped to 0; values > 1 are clamped to 1. Reason tags `nonfinite_gate`, `gate_below_zero`, `gate_above_one`.

`protectState` is invoked with a phase tag identifying its caller (`calculateCurrent`, `solveODE_start`, `solveODE_end`, `derivatives`). All corrections at a given (phase, variable, reason) triple are bucketed into a single `ProtectionSummaryEntry` that tracks the total count, the first correction time and integration point, and the min/max original values and the maximum absolute delta. The summary is written on exit to `TNNP_numericalProtection_summary.dat`.

9 Activation tracking and benchmarks

9.1 Activation time per cell

Once a cell crosses the activation threshold `activationThreshold` (V_{thr}) for the first time in a given time step, the activation time is recorded by linear interpolation of V across the step,

$$w = \text{clip}\left(\frac{V_{\text{thr}} - V^n}{V^{n+1} - V^n}, 0, 1\right), \quad t_{\text{act}} = t^n + w \Delta t, \quad (28)$$

implemented in `updateActivationTimes`. A boolean flag `calculateActivationTime[cellI]` is cleared after the first crossing, so subsequent re-crossings (e.g. during recovery) are ignored.

9.2 Early termination on benchmark point

When `stopAfterPointActivation` is true, the solver identifies the cell nearest to `stopActivationPoint` (P8 in the Niederer benchmark). As soon as that cell records a non-zero t_{act} , the variable `effectiveEndTime` is set to $t_{\text{act}} + \text{stopDelayAfterActivation}$, terminating the simulation early once the wave has fully developed.

9.3 Diagonal sampling

Post-processing computes activation times at `nDiagonalSamples` points uniformly distributed between `diagonalStart` and `diagonalEnd` by inverse-distance-weighted interpolation (`sampleIDW`):

$$t_{\text{act}}(\mathbf{x}) = \frac{\sum_{j \in \mathcal{N}_k(\mathbf{x})} w_j t_{\text{act},j}}{\sum_{j \in \mathcal{N}_k(\mathbf{x})} w_j}, \quad w_j = \frac{1}{\|\mathbf{x} - \mathbf{C}_j\|^p}, \quad (29)$$

where $\mathcal{N}_k$ is the set of `sampleNeighbours` nearest cells and $p = \text{samplePower}$. Default $k = 8$, $p = 2$.

10 Solver flow chart

The pseudo-code below summarises the per-time-step flow of the solver and shows the `run.convergence.py` `constants` / `dict keys` that control each branch.

Algorithm 1 High-level time-step loop of highOrderElectroActivationFoamImplicitJfNK.PETSc.

```
1: PetscInitialize (after setRootCaseLists so OpenFOAM owns MPI).
2: Read dictionaries.
3: Assemble PETSc Mats: M (massMatrix), K (useHighOrder_Vm), L = K/( $\chi C_m$ ).
4:  $V^0 \leftarrow -84$  mV (initial condition).
5:  $V^{-1} \leftarrow V^0$ ,  $\Delta t_{\text{prev}} \leftarrow 0$ , hasPrev  $\leftarrow$  false.
6: while  $t^n < \text{END\_TIME}$  do
7:    $\Delta t \leftarrow \text{DT\_VALUES}$  (current sweep value).
8:   Build A = M/ $\Delta t$  -  $\theta$ L, B = M/ $\Delta t$  + (1 -  $\theta$ )L via MatDuplicate + MatScale + MatAXPY
   (implicitScheme  $\rightarrow \theta$ ).
9:   Apply stimulus to  $I_{\text{ext}}$  (stimulusProtocol).
10:  if jfnkInitGuessOrder  $\geq 1$  and hasPrev then
11:     $V_{\text{guess}} \leftarrow V^n + (\Delta t / \Delta t_{\text{prev}}) (V^n - V^{n-1})$ , clamp to [jfnkInitGuessVmMin, jfnkInitGuessVmMax].
12:  else
13:     $V_{\text{guess}} \leftarrow V^n$ .
14:  end if
15:   $V^{n-1} \leftarrow V^n$ ,  $\Delta t_{\text{prev}} \leftarrow \Delta t$ , hasPrev  $\leftarrow$  true.
16:  if nonlinearMethod = JFNK then
17:     $x \leftarrow V_{\text{guess}}$ .
18:    for  $k = 0, 1, \dots, \text{IMPLICIT\_NONLINEAR\_ITERATIONS}-1$  do
19:       $F_k \leftarrow \text{COMPUTE\_RESIDUAL}(x)$  (advances TNNP via ODE_REL_TOL, ODE_ABS_TOL; clamp Vm
      by jfnkClampODEInput).
20:      if all convergence tests pass (nonlinearTolerance, ...) then
21:        break
22:      end if
23:      Build a PETSc MatShell wrapping the FD Jacobian-vector product  $Jv \approx (F(x + \varepsilon v) - F_k)/\varepsilon$  with  $\varepsilon$  from jfnkEpsilon.
24:      Solve  $J\delta = -F_k$  with KSPGMRES + MGS (jfnkMaxKrylovIterations, jfnkLinearTolerance,
      jfnkMaxRestarts).
25:      if jfnkLineSearch then
26:        Armijo backtracking (jfnkArmijoC, jfnkLineSearchMaxIter, jfnkLineSearchAlphaMin):
        find  $\alpha$  satisfying  $\|F(x + \alpha\delta)\|^2 \leq (1 - 2c\alpha)\|F_k\|^2$ .
27:      else
28:         $\alpha \leftarrow \text{nonlinearRelaxation}$ .
29:      end if
30:       $x \leftarrow x + \alpha\delta$ .
31:    end for
32:     $V^{n+1} \leftarrow x$ .
33:  else if nonlinearMethod = picard or diagonalLion then
34:    for  $k = 0, 1, \dots, \text{IMPLICIT\_NONLINEAR\_ITERATIONS}-1$  do
35:      Solve  $AV_* = r(I_{\text{ion}}(x_k))$  with solveLinearKSP (linearSolver, preconditioner, tolerance).
36:       $x_{k+1} \leftarrow (1-\omega)x_k + \omega V_*$ ,  $\omega = \text{nonlinearRelaxation}$ .
37:      if convergence tests pass then break
38:    end if
39:  end for
40:   $V^{n+1} \leftarrow x_{k+1}$ .
41: end if
42: if not converged and not nonlinearAcceptUnconverged then
43:   Warn; roll back:  $V^{n+1} \leftarrow V^n$ ,  $I_{\text{ion}}^{n+1} \leftarrow I_{\text{ion}}^n$ ,  $s^{n+1} \leftarrow s^n$ .
44: end if
45: Record activation times (activationThreshold), P8 stop check (stopAfterPointActivation,
stopDelayAfterActivation).
46: Write nonlinear-residual log if writeNonlinearResiduals.
47:  $t^{n+1} \leftarrow t^n + \Delta t$ .
48: end while
```

11

```
49: Post-process: IDW sampling at P8 and diagonal (nDiagonalSamples, sampleNeighbours, samplePower).
50: PetscFinalize (after the inner { } scope has destroyed all PETSc Mats/Vecs).
```

Legend. Blue = Python constant set in `run_convergence.py`; green = OpenFOAM dictionary key read by the solver. Constants and keys are mapped one-to-one in section 11.

11 Driver script: `run_convergence.py`

The Python driver `run_convergence.py` writes the OpenFOAM dictionaries and runs the solver in a parameter sweep. Every solver knob is exposed as a top-level uppercase constant. The tables below group the constants by topic; *Key* is the dictionary entry emitted by the script, and *Section* points to the location in this report where the underlying algorithm is described.

11.1 Geometry and parameter sweep

Python constant	OpenFOAM key	Meaning
DIMENSIONS	—	List of dimensions to sweep (2D, 3D).
DX_VALUES_MM	<code>blockMeshDict</code> cell counts	Uniform cell size in millimetres; mesh is generated by <code>blockMesh</code> from these.
DT_VALUES	<code>controlDict.deltaT</code>	Outer time-step sizes to sweep (seconds).
END_TIME	<code>controlDict.endTime</code>	Final simulation time (seconds).
CFL	<code>timeIntegrationProperties.CFL</code>	Informational reference Courant number; not enforced by the implicit solver.
ACTIVE_TIME_SCHEMES	<code>spatialIntegrationProperties.subImplicitScheme</code>	Subset of {backwardEuler, crankNicolson}.
ACTIVE_MASS_MATRICES	<code>spatialIntegrationProperties.subMassMatrix</code>	Subset of {lumped, consistent}.
ACTIVE_HO_PAIRS_Vm_States	<code>useHighOrder_Vm</code> <code>useHighOrder_Iion</code> <code>LRECoeffs_*</code>	/ Tuples (Vm-level, Iion-level); + level is one of N0, p1, p2, p3.
ACTIVE_NONLINEAR_METHODS	<code>implicitHOCoeffs.nonlinearMethods</code>	Subset of {JFNK, picard, diagonalIion, ...}.
ACTIVE_STATE_ODE_SOLVERS	<code>timeIntegrationProperties.solvers</code>	Subset of OpenFOAM ODE solver names.
SIGMA_X, SIGMA_Y, SIGMA_Z	<code>cardiacProperties</code>	Diagonal components of the conductivity tensor D .
CHI	<code>cardiacProperties.chi</code>	Surface-to-volume ratio.
CM	<code>cardiacProperties.Cm</code>	Membrane capacitance per area.

11.2 Nonlinear solver (JFNK / Picard)

Python constant	OpenFOAM key	Meaning
IMPLICIT_NONLINEAR_ITERATIONS	<code>nonlinearIterations</code>	Hard cap on Newton/Picard outer iterations per time step.
IMPLICIT_MIN_NONLINEAR_ITERATIONS	<code>minNonlinearIterations</code>	Minimum iterations before the convergence test is allowed to terminate the loop.
NONLINEAR_TOLERANCE	<code>nonlinearTolerance</code>	Tolerance on the coupled residual $\ F\ /\ x\ $. Cannot be below the ODE <code>relTol</code> floor.

Python constant	OpenFOAM key	Meaning
NONLINEAR_VM_TOLERANCE	nonlinearVmTolerance	Relative L^2 change in V_m between successive iterates.
NONLINEAR_IION_TOLERANCE	nonlinearIionTolerance	Same metric on the I_{ion} field.
NONLINEAR_STATES_TOLERANCE	nonlinearStatesTolerance	Maximum over state index of relative L^2 change in each ionic state.
NONLINEAR_REQUIRE_STATES_CONVERGENCE	nonlinearRequireStatesConvergence	If true, the states test is required in addition to the others.
NONLINEAR_RELAXATION	nonlinearRelaxation	Initial Newton step length α_0 (also the Picard relaxation factor in non-JFNK branches).

11.3 JFNK matrix-free options

Python constant	OpenFOAM key	Meaning
JFNK_MAX_KRYLOV_ITERATIONS	jfnkMaxKrylovIterations	GMRES restart cycle m .
JFNK_LINEAR_TOLERANCE	jfnkLinearTolerance	Relative tolerance of the inner KSP.
JFNK_EPSILON	jfnkEpsilon	Forward-difference perturbation ε_0 in (20). Set near $\sqrt{\eta}$, where η is the ODE relTol.
DIAGONAL_IION_EPSILON	diagonalIionEpsilon	Step size for the diagonal $\partial I_{\text{ion}} / \partial V_m$ in the diagonalIion branch.

11.4 Linear solver

Python constant	OpenFOAM key	Meaning
IMPLICIT_LINEAR_SOLVER	linearSolver	PETSc KSP type used by the explicit-operator solve (Picard / diagonal branches).
IMPLICIT_TOLERANCE	tolerance	Relative residual tolerance of the explicit-operator KSP.
IMPLICIT_MAX_ITERATIONS	maxIterations	Iteration cap on the explicit-operator KSP.

11.5 Inner ODE integrator (RK45)

Python constant	OpenFOAM key	Meaning
ODE_ABS_TOL	timeIntegrationProperties.absoluteTol	Absolute error tolerance per state.
ODE_REL_TOL	timeIntegrationProperties.relativeTol	Relative error tolerance per state. Drives the noise floor of the matrix-free Jacobian.
ODE_INITIAL_STEP	initialODEStep	Initial sub-step size hint.

Python constant	OpenFOAM key	Meaning
ODE_MAX_STEPS	maxSteps	Cap on RKF45 sub-steps per outer call. Above this the solver raises a fatal error.

11.6 LRE high-order parameters per p-level

P_LEVELS_BY_DIM[dim][p] is a dictionary giving, for each combination of geometric dimension and polynomial order:

Sub-key	Meaning	Typical value
N	Polynomial order (1, 2 or 3)	matches the p tag
Nn	Number of stencil neighbours required	13–60 depending on (dim, p)
k	Stencil search radius (in cell diameters)	6
maxStencilSize	Hard cap on stencil size	33–80

The script writes these into a nested `highOrderCoeffs{LRECoeffs.Vm{...}; LRECoeffs.Iion{...}}` block in `spatialIntegrationProperties` whenever HO is enabled for either Vm or I_{ion} .

11.7 Post-processing and run-control

Python constant	OpenFOAM key	Meaning
ACTIVATION_THRESHOLD	activationThreshold	V_{thr} for activation-time crossing (V).
N_DIAGONAL_SAMPLES	nDiagonalSamples	Number of sample points along the diagonal.
SAMPLE_NEIGHBOURS	sampleNeighbours	Number of nearest cells used by IDW.
SAMPLE_POWER	samplePower	Inverse-distance power p .
STOP_AFTER_POINT_ACTIVATION	stopAfterPointActivation	Terminate after the P8 cell activates.
STOP_DELAY_AFTER_ACTIVATION	stopDelayAfterActivation	Wall-clock-independent delay after P8 activation before stopping (seconds).
WRITE_NONLINEAR_RESIDUALS	writeNonlinearResiduals	Emit per-iteration residual log to <code>postProcessing/.../nonlinearResiduals.d</code>
SAVE_LOGS_ON_SUCCESS	—	Python-only flag; if <code>false</code> , deletes solver and blockMesh log files after a successful run.

A Dimension table

OpenFOAM dimension vectors (mass, length, time, temperature, amount, current, luminous intensity) of the fields declared in `createFields.H`:

Field	OpenFOAM dimensionSet	SI
Vm	[1, 2, -3, 0, 0, -1, 0]	volt
Iion	dimVoltage/dimTime	V s ⁻¹
externalStimulusCurrent	dimCurrent/dimVolume	A m ⁻³
conductivity	dimCurrent^2 dimTime^3 / (dimMass dimVolume)	S m ⁻¹
chi	dimless/dimLength	m ⁻¹
Cm	dimCurrent dimTime / (dimVoltage dimArea)	F m ⁻²

B File map

Path (relative to cardiacFoam-pc/)	Role
applications/solvers/.../JfNK_PETSc.C	Main translation unit; time loop, JFNK loop, KSP wrapper.
applications/solvers/.../createFields.H	Field instantiation and dictionary reading.
src/ionicModels/TNNP/TNNP.H, .C	TNNP cell model and protection guards.
src/ionicModels/TNNP/TNNP_2004.H, *Names.H	Generated CellML kernel and state-name table.
src/ionicModels/ionicModel/ionicModel.H,C	Base class with run-time selection of the ODE solver.
tutorials_electro/.../run_convergence.py	Python driver of the parameter sweep.
constant/cardiacProperties	χ , C_m , conductivity tensor, ionic model selection.
constant/timeIntegrationProperties	ODE solver, tolerances, sub-step cap.
constant/spatialIntegrationProperties	Time scheme, HO flags, JFNK / KSP / line-search options.
constant/stimulusProtocol	Stimulus boxes, intensity, duration, start times.

References

- [1] K. H. W. J. ten Tusscher, D. Noble, P. J. Noble, A. V. Panfilov, *A model for human ventricular tissue*, American Journal of Physiology – Heart and Circulatory Physiology **286** (2004) H1573–H1589.
- [2] S. A. Niederer et al., *Verification of cardiac tissue electrophysiology simulators using an N-version benchmark*, Progress in Biophysics and Molecular Biology **104** (2012) 22–48.
- [3] D. A. Knoll, D. E. Keyes, *Jacobian-Free Newton-Krylov methods: a survey of approaches and applications*, Journal of Computational Physics **193** (2004) 357–397.
- [4] J. Nocedal, S. J. Wright, *Numerical Optimization*, 2nd edition, Springer, 2006 (Chapter 3, line-search methods).
- [5] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd edition, SIAM, 2003.
- [6] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, Springer, 1996.